

mksglu/context-mode

Context Mode : l'outil qui empêche les assistants de programmation de perdre la mémoire, à essayer en interne, pas à bâtir dessus

Context window optimization for AI coding agents. Sandboxes tool output (98% reduction), persists session memory, and enforces routing across 17 platforms via MCP + hooks.

21 624

ÉTOILES GITHUB

1 555

FORKS

10

CONTRIBUTEURS ACTIFS
sur 12 semaines

23 févr. 2026

CRÉÉ LE

Que fait ce projet, en une phrase, et pour qui ?

“

Context Mode promet jusqu'à 98 % de mémoire économisée aux assistants de code, sur 17 plateformes.

Les assistants de programmation par IA ont une mémoire de travail limitée, et chaque outil qu'ils appellent la remplit de données brutes ; Context Mode filtre ces données à la source et annonce jusqu'à 98 % d'économie. C'est comme un assistant qui ne vous apporte que le résumé d'un dossier de 500 pages, et qui garde le dossier sous la main si vous voulez un détail.

Le public : les équipes qui utilisent un assistant de code comme Claude Code, Codex ou Copilot, sur 17 plateformes annoncées. Le README affiche des logos d'entreprises utilisatrices (Microsoft, Google, Stripe...) ; ces usages ne sont pas vérifiables dans les données relevées, à vérifier dans <https://github.com/mksglu/context-mode#readme>.

Est-ce que ça marche et qui s'en sert ?

“

21 624 étoiles en un peu plus de six mois : le marché a répondu, mais 119 demandes attendent encore une réponse.

Signal d'adoption : 21 624 étoiles et 1 555 copies (forks) sur GitHub, pour un projet né le 2026-02-23, soit en un peu plus de six mois. Le README revendique une première place sur Hacker News.

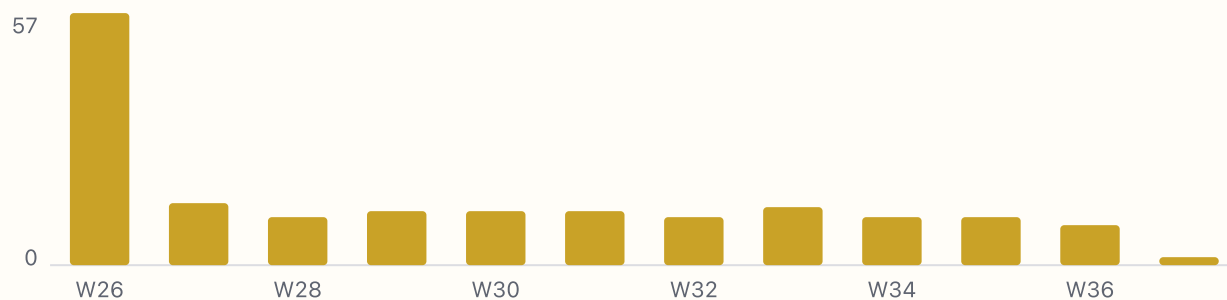
Livraisons : 10 versions publiées entre le 2026-06-01 et le 2026-06-29, de la v1.0.160 à la v1.0.169. Aucune version relevée après fin juin ; à vérifier dans <https://github.com/mksglu/context-mode/releases>.

Retours : 119 demandes ouvertes, 6 résolues sur les 30 derniers jours. La demande la plus commentée (154 commentaires) cherche des beta-testeurs sur 15 plateformes et 3 systèmes. Ça marche assez pour attirer, pas assez pour que tout soit traité.

Le projet est-il vivant ?

“

Vivant mais au ralenti : de 57 modifications par semaine fin juin à 9 début septembre, avec 107 propositions en attente.



Oui, mais il ralentit. Le rythme est passé de 57 modifications en semaine 26 à environ 11 par semaine tout l'été, puis 9 en semaine 36. Dernière modification le 2026-09-08.

Le stock de travail en attente grossit : 107 propositions de changement ouvertes, aucune acceptée sur 30 jours dans les données relevées. Image : un guichet où la file s'allonge parce qu'il n'y a qu'un guichetier.

Qui est derrière ?

“

Un projet à une seule personne : 60 modifications sur 12 semaines pour l'auteur, 6 pour le suivant.



Une personne : Mert Koseoğlu, compte `mksglu`, 60 modifications sur 12 semaines, contre 6 pour le deuxième et 1 ou 2 pour les huit suivants. L'auteur se présente lui-même comme mainteneur solo avec peu de temps.

C'est un compte personnel, pas une organisation. Il accepte les dons via GitHub Sponsors. Pas de politique de sécurité ni de liste de responsables par zone.

Quels sont les trois risques ?

“

Le risque numéro un est humain : tout repose sur une seule personne, et la licence interdit d'en faire un service payant.

-
- **license** Licence non standard « Elastic License 2.0 (ELv2) » : lire LICENSE avant tout usage.
 - **security** Pas de SECURITY.md : aucun canal déclaré pour signaler une faille.
 - **bus_factor** Un seul contributeur porte la moitié des commits sur 12 semaines.
 - **ci** 5 workflows dans .github/workflows ; ci.yml (seul lu) : typecheck, build, bundle, vitest sur ubuntu, macos et windows, déclenché par workflow_dispatch, push et pull_request sur main et next.
 - **deps** 15 dépendances déclarées dans package.json (8 runtime, 7 dev) ; better-sqlite3 est un module natif externalisé du bundle ; node >= 22.5.0 requis.
-

1. Dépendance à une seule personne, niveau élevé : si l'auteur s'arrête, le projet s'arrête.
2. Licence non standard, niveau moyen : Elastic License 2.0, qui interdit de revendre le logiciel comme service hébergé. Utilisable en interne, pas comme produit.
3. Sécurité, niveau moyen : aucun canal déclaré pour signaler une faille, sur un outil qui exécute du code et lit vos sessions.

Les deux autres risques suivis, chaîne de livraison et dépendances, sont faibles.

Qu'en fait-on ?

“

Un essai interne sur une équipe, pas un produit : la licence et la dépendance à une personne l'interdisent pour l'instant.

Le carnet de demandes est peu structuré : un seul thème étiqueté, « enhancement », 6 demandes, et aucun jalon planifié. Les 10 propositions de changement les plus récentes sont des corrections ciblées (délais, routage réseau, budgets par plateforme).

Lecture : un outil utile et populaire, porté par un seul développeur, sous une licence qui bloque toute revente en service. Décision raisonnable : l'essayer en interne sur une équipe, sans en dépendre pour un produit, et surveiller le rythme des versions à trois mois.

Ce que je retiens

- Un vrai problème, une vraie réponse : jusqu'à 98 % de mémoire économisée pour les assistants de code, 21 624 étoiles en un peu plus de six mois.
- Une seule personne derrière : 60 modifications sur 12 semaines contre 6 pour le suivant, et 107 propositions de changement en attente.
- Une licence qui autorise l'usage interne et interdit la revente comme service hébergé.
- Le rythme baisse : 57 modifications par semaine fin juin, 9 début septembre, et aucune version publiée après le 29 juin dans les données relevées.
- Décision : essai interne sur une équipe volontaire, aucune dépendance produit, et revue du rythme des versions avant tout engagement plus large.